Day 9 – DSA Core I: Arrays, Strings, Hashing & Prefix Sums

Today's goal is not to memorize individual problems. It is to recognize a small set of reusable patterns.

Day 9 Mental Model

When you see an array or string problem, ask:

1. **Can I solve it with one traversal?**
2. **Do I need to remember previously seen values?** Use a `set` or `dict`.
3. **Do I need counts?** Use a frequency dictionary.
4. **Do I need repeated range sums?** Use a prefix sum.
5. **Am I searching for a partner value?** Store previously seen values and search for the complement.

A useful memory phrase is:

> **Scan → Store → Look up → Update**

---

1. Complexity Basics

Concept

Big-O describes how runtime or memory grows as input size n grows.

Complexity	Meaning	Common example
`O(1)`	Constant work	Access `nums[i]`
`O(n)`	Process every item once	Traverse an array
`O(n log n)`	Usually sorting	`sorted(nums)`
$O(n^2)$	Compare many pairs	Nested loops
`O(n)` space	Store up to n values	Hash set or prefix array

In interviews, separate:

- **Time complexity:** how much work is performed.
- **Space complexity:** how much additional memory is used.

Important Python Operations

Operation	Average complexity
`nums[i]`	`O(1)`
`nums.append(x)`	`O(1)` amortized
`x in list`	`O(n)`
`x in set`	`O(1)` average
`key in dict`	`O(1)` average
`dict[key] = value`	`O(1)` average
Sorting	`O(n log n)`
String slicing `text[a:b]`	`O(k)`
List slicing `nums[a:b]`	`O(k)`

Here, k is the size of the copied slice.

Example

```
def contains_value(nums: list[int], target: int) -> bool:
    """Return True when target exists in nums."""

    for value in nums:  # At most n iterations
        if value == target:
            return True

    return False
```

- Time: `O(n)`
- Extra space: `O(1)`

Pitfalls

Candidates commonly:

- Say dictionary lookup is always `O(1)`. It is **average-case `O(1)`**.
- Ignore additional memory used by dictionaries and sets.
- Forget that Python slicing creates a copy.
- Build strings repeatedly using `result += character` inside a large loop. This can become expensive because strings are immutable.
- Optimize before first explaining the simple brute-force solution.

---

2. Arrays and Strings

Concept

An **array** is an ordered sequence of values.

A Python `list` behaves like a dynamic array.

A **string** is an ordered sequence of characters. Python strings are immutable, meaning individual characters cannot be changed directly.

```
text = "Disney"

# Invalid:
# text[0] = "d"

# Create a new string instead:
updated = "d" + text[1:]
```

Arrays and strings use many of the same patterns:

- Traversal
- Index-based comparison
- Frequency counting
- Subarray or substring processing
- Prefix calculations

---

3. Traversal Pattern

Concept

Traversal means visiting elements, usually from left to right.

Common traversal styles:

```
for value in nums:
    ...
```

Use this when only the value matters.

```
for index, value in enumerate(nums):
    ...
```

Use this when both the position and value matter.

```
for index in range(len(nums)):
    ...
```

Use this when you need complete index control.

Example: Count Failed Events

Imagine that an AI platform emits event statuses.

```
def count_failed_events(statuses: list[str]) -> int:
    """Count how many events have a failed status."""

    failed_count = 0

    for status in statuses:
        if status == "failed":
            failed_count += 1

    return failed_count


statuses = ["success", "failed", "success", "failed"]
print(count_failed_events(statuses))  # 2
```

- Time: `O(n)`
- Extra space: `O(1)`

Example: Compare Adjacent Values

Suppose you want to detect sudden increases in token usage.

```
def count_token_spikes(token_counts: list[int]) -> int:
    """
    Count positions where the current token count is greater
    than the previous token count.
    """

    spikes = 0

    # Start at index 1 because index 0 has no previous element.
    for index in range(1, len(token_counts)):
        if token_counts[index] > token_counts[index - 1]:
            spikes += 1

    return spikes


counts = [100, 130, 110, 180]
print(count_token_spikes(counts))  # 2
```

- Time: `O(n)`
- Extra space: `O(1)`

Pitfalls

- Starting at index `0` while accessing `index - 1`.
- Accessing `nums[index + 1]` at the final index.
- Forgetting empty-array behavior.
- Modifying an array while traversing it.
- Using indexes when direct value traversal would be simpler.
- Accidentally writing two traversals when one is enough.

4. Subarrays and Substrings

Concept

A **subarray** is a continuous part of an array.

For:

```
[10, 20, 30]
```

Valid subarrays include:

```
[10]
[20]
[30]
[10, 20]
[20, 30]
[10, 20, 30]
```

`[10, 30]` is not a subarray because the elements are not contiguous.

A **substring** is the same idea for strings.

For `"abc"`:

```
"a", "b", "c", "ab", "bc", "abc"
```

`"ac"` is not a substring.

A **subsequence**, in contrast, does not need to be contiguous.

Number of Subarrays

An array of length n contains:

$$\frac{n(n+1)}{2}$$

subarrays.

This is why generating every subarray normally requires $O(n^2)$ work.

Example: Generate All Subarrays

```
def generate_subarrays(nums: list[int]) -> list[list[int]]:
    """Return every contiguous subarray."""

    result: list[list[int]] = []

    for start in range(len(nums)):
        current: list[int] = []

        # Every end position from start onward creates one subarray.
        for end in range(start, len(nums)):
            current.append(nums[end])

            # Copy current because it will continue changing.
            result.append(current.copy())

    return result


print(generate_subarrays([1, 2, 3]))
```

Output:

```
[[1], [1, 2], [1, 2, 3], [2], [2, 3], [3]]
```

- Time: at least $O(n^2)$, and more when accounting for copied output
- Space: $O(n^2)$ or more for storing all subarrays

Real System Use Cases

Subarray-style problems appear in:

- Request volume during a continuous time window
- Token usage over consecutive requests
- Errors during consecutive deployment stages
- Revenue or cost over consecutive days
- CPU utilization during a particular interval

Pitfalls

- Confusing a subarray with a subsequence.
- Forgetting that the range end is exclusive in Python.
- Creating slices repeatedly without considering copying costs.
- Using brute-force generation when only a sum or count is needed.
- Assuming all subarray problems require nested loops. Prefix sums, sliding windows, and hashing often improve them.

---

5. Prefix Sum Basics

Concept

A prefix sum stores cumulative totals.

For:

```
nums = [2, 4, 1, 3]
```

Use a prefix array with one extra leading zero:

```
prefix = [0, 2, 6, 7, 10]
```

Definition:

```
prefix[i] = sum of nums[0:i]
```

Therefore:

```
prefix[0] = 0
prefix[1] = nums[0]
prefix[2] = nums[0] + nums[1]
```

The sum from index `left` to `right`, inclusive, is:

```
prefix[right + 1] − prefix[left]
```

For `left = 1`, `right = 3`:

```
4 + 1 + 3 = 8
```

Using prefix sums:

```
prefix[4] − prefix[1]
= 10 − 2
= 8
```

Why Use a Leading Zero?

Without a leading zero, ranges starting at index `0` require special handling.

With the zero:

```
sum from 0 to right = prefix[right + 1] − prefix[0]
```

The same formula works for every valid range.

Example: Build and Query Prefix Sums

```
def build_prefix_sum(nums: list[int]) -> list[int]:
    """
    Build a prefix sum where prefix[i] contains the sum
    of the first i elements.
    """

    prefix = [0] * (len(nums) + 1)

    for index, value in enumerate(nums):
        # prefix[index] already contains the sum before this value.
        prefix[index + 1] = prefix[index] + value

    return prefix


def range_sum(prefix: list[int], left: int, right: int) -> int:
    """
    Return the sum from left to right, inclusive.

    The original array length is len(prefix) - 1.
    """

    array_length = len(prefix) - 1

    if left < 0 or right >= array_length or left > right:
        raise ValueError("Invalid range")

    # prefix[right + 1] contains everything through nums[right].
    # Subtract everything before nums[left].
    return prefix[right + 1] - prefix[left]


nums = [2, 4, 1, 3]
prefix = build_prefix_sum(nums)

print(prefix)                 # [0, 2, 6, 7, 10]
print(range_sum(prefix, 1, 3))  # 8
```

Complexity

Building the prefix sum:

- Time: $O(n)$
- Space: $O(n)$

Each range query:

- Time: $O(1)$
- Extra space: $O(1)$

Without prefix sums, each query may require $O(n)$ time.

Example: Token Usage Over a Request Range

```
token_counts = [100, 250, 80, 170, 300]
prefix = build_prefix_sum(token_counts)

# Total tokens used by requests at indexes 1, 2, and 3.
print(range_sum(prefix, 1, 3))  # 500
```

This pattern could support:

- Token usage dashboards
- API request counts per time interval
- Cost calculation across consecutive requests
- Error counts over a deployment window

When Prefix Sums Are Useful

Use prefix sums when:

- The underlying array is mostly unchanged.
- You have multiple range-sum queries.
- You need cumulative totals.
- Negative numbers may exist and a normal sliding-window approach is unsuitable.

Do not automatically use them for a single range query. A direct traversal may be simpler.

Pitfalls

- Using `prefix[right] - prefix[left]` instead of `prefix[right + 1] - prefix[left]`.
- Forgetting that `right` is inclusive.
- Creating a prefix array of length n instead of `n + 1`.
- Forgetting the initial zero.
- Rebuilding the prefix sum for every query.
- Using a static prefix sum when the original array changes frequently. Updates would make later prefix values stale.
- In languages with fixed-size integers, ignoring overflow for very large sums.

---

6. Hashing with Dictionaries and Sets

Concept

Hashing provides fast average-case lookup.

Python provides:

- `dict`: stores key-value pairs.
- `set`: stores unique values.

Use a dictionary when you need associated information:

```
event_count["failed"] = 5
```

Use a set when you only need to know whether something exists:

```
processed_event_ids.add("evt-123")
```

Core Decision

Ask:

> Do I need only existence, or do I need additional information?

- Existence only → `set`
- Count, index, metadata, mapping → `dict`

---

7. Frequency Counting Pattern

Concept

Frequency counting records how many times each value appears.

Example:

```
events = ["success", "failed", "success", "timeout"]
```

Frequency map:

```
{
    "success": 2,
    "failed": 1,
    "timeout": 1
}
```

Example

```
def frequency_count(items: list[str]) -> dict[str, int]:
    """Count how many times each item appears."""

    frequencies: dict[str, int] = {}

    for item in items:
        # If item is absent, get returns 0.
        frequencies[item] = frequencies.get(item, 0) + 1

    return frequencies


events = ["success", "failed", "success", "timeout"]
print(frequency_count(events))
```

- Time: `O(n)` average
- Space: `O(k)`

Here, `k` is the number of distinct values and can be as large as `n`.

Real System Example: Count User Actions

```
def count_user_actions(actions: list[str]) -> dict[str, int]:
    """Count user actions for an analytics pipeline."""

    action_counts: dict[str, int] = {}

    for action in actions:
        action_counts[action] = action_counts.get(action, 0) + 1

    return action_counts


actions = ["login", "search", "search", "purchase", "logout"]
print(count_user_actions(actions))
```

Possible output:

```
{
    "login": 1,
    "search": 2,
    "purchase": 1,
    "logout": 1
}
```

Real GenAI Example: Count Tokens

For interview learning, suppose tokens have already been produced by a tokenizer:

```
def count_tokens(tokens: list[str]) -> dict[str, int]:
    """Count token occurrences."""

    counts: dict[str, int] = {}

    for token in tokens:
        counts[token] = counts.get(token, 0) + 1

    return counts


tokens = ["the", "model", "generated", "the", "answer"]
print(count_tokens(tokens))
```

Alternative: `Counter`

Python provides `collections.Counter`:

```
from collections import Counter

counts = Counter(["a", "b", "a"])
print(counts)  # Counter({'a': 2, 'b': 1})
```

In an interview, implementing the dictionary approach first demonstrates that you understand the pattern.

Pitfalls

- Writing:

```
counts[item] += 1
```

before initializing the key.

- Forgetting normalization:

```
"Error", "error", and "ERROR"
```

may need to represent the same event.

- Counting whole strings when tokenization is required.
- Using a set when counts are needed.
- Assuming dictionary iteration is sorted by frequency.
- Forgetting that dictionary keys must be hashable. Lists cannot be dictionary keys.

---

8. Two-Sum Style Pattern

Concept

Two Sum asks:

> Are there two values whose sum equals a target?

Example:

```
nums = [2, 7, 11, 15]
target = 9
```

For each value:

```
required partner = target − current value
```

When current value is 7:

```
required partner = 9 − 7 = 2
```

If 2 has already been seen, the answer is found.

This is called the **complement pattern**.

Brute Force

Compare every pair:

```
for i in range(n):
    for j in range(i + 1, n):
        ...
```

- Time: $O(n^2)$
- Space: $O(1)$

Optimized Hashing Solution

```
def two_sum(nums: list[int], target: int) -> list[int]:
    """
    Return indexes of two different elements whose sum equals target.
    Return an empty list when no pair exists.
    """

    # Maps number -> its previously seen index.
    seen: dict[int, int] = {}

    for index, value in enumerate(nums):
        complement = target - value

        # Check before inserting the current value.
        # This prevents using the same array element twice.
        if complement in seen:
            return [seen[complement], index]

        seen[value] = index

    return []


print(two_sum([2, 7, 11, 15], 9))  # [0, 1]
```

- Time: $O(n)$ average
- Space: $O(n)$

Real System Analogy

Suppose a batch has a capacity of 100 units. You want to identify two pending tasks whose resource requirements exactly fill the batch.

```
task sizes = [30, 70, 45, 20]
capacity = 100
```

When processing 70, search for 30.

This is conceptually the same as Two Sum.

Pitfalls

Pitfall 1: Using the same element twice

For:

```
nums = [3]
target = 6
```

You cannot use index `0` twice.

Checking before inserting avoids this.

Pitfall 2: Returning values instead of indexes

Read the question carefully. Some versions request:

- Indexes
- Values
- Boolean existence
- Number of pairs

Pitfall 3: Sorting without considering index requirements

Sorting makes two pointers possible, but it changes the original positions unless indexes are stored.

Pitfall 4: Mishandling duplicates

For:

```
nums = [3, 3]
target = 6
```

The hashing solution works because the first 3 is stored before processing the second.

Pitfall 5: Continuing after finding the answer

When only one answer is required, return immediately.

---

9. Detecting Duplicates

Concept

A duplicate exists when a value appears more than once.

A set is ideal because it stores unique values and provides fast membership checks.

Example

```
def contains_duplicate(nums: list[int]) -> bool:
    """Return True when any value appears more than once."""

    seen: set[int] = set()

    for value in nums:
        if value in seen:
            return True

        seen.add(value)

    return False


print(contains_duplicate([1, 2, 3, 1]))  # True
print(contains_duplicate([1, 2, 3]))     # False
```

- Time: `O(n)` average
- Space: `O(n)`

Real System Example: Duplicate Event IDs

Distributed systems may deliver an event more than once.

```
def has_duplicate_event_id(event_ids: list[str]) -> bool:
    """Detect whether an event ID was received more than once."""

    processed: set[str] = set()

    for event_id in event_ids:
        if event_id in processed:
            return True

        processed.add(event_id)

    return False
```

This relates to:

- Idempotency
- Duplicate message detection
- Request deduplication
- Preventing repeated model jobs
- Avoiding double billing or double processing

In production, an in-memory set only works within one process and for a limited lifetime. A distributed system may require Redis, a database, or another shared idempotency store.

Alternative Solution

```
def contains_duplicate_short(nums: list[int]) -> bool:
    return len(nums) != len(set(nums))
```

This is concise, but the explicit traversal is often better for explaining the interview pattern.

Pitfalls

- Using a list for `seen`, which produces $O(n^2)$ worst-case behavior.
- Returning only after the entire traversal when a duplicate was already found.
- Forgetting that a set does not retain duplicate counts.
- Assuming an in-memory set provides distributed deduplication.
- Using mutable objects such as lists as set elements.

---

10. Anagram Detection

Concept

Two strings are anagrams when they contain the same characters with the same frequencies.

Examples:

```
"listen" and "silent" → anagrams
"rat" and "car"       → not anagrams
"aab" and "abb"       → not anagrams
```

Anagram checking is a frequency-comparison problem.

Hashing Solution

```
def is_anagram(first: str, second: str) -> bool:
    """Return True when first and second are anagrams."""

    if len(first) != len(second):
        return False

    counts: dict[str, int] = {}

    # Count characters in the first string.
    for character in first:
        counts[character] = counts.get(character, 0) + 1

    # Remove the corresponding counts using the second string.
    for character in second:
        if character not in counts:
            return False

        counts[character] -= 1

        # A negative count means second contains this character
        # more times than first.
        if counts[character] < 0:
            return False

    return True


print(is_anagram("listen", "silent"))  # True
print(is_anagram("rat", "car"))        # False
```

- Time: `O(n)`
- Space: `O(k)`

Here, k is the number of distinct characters.

Simpler Frequency Comparison

```
def is_anagram_using_two_maps(first: str, second: str) -> bool:
    """Check anagrams by comparing two frequency dictionaries."""

    if len(first) != len(second):
        return False

    return frequency_count(list(first)) == frequency_count(list(second))
```

Sorting Solution

```
def is_anagram_by_sorting(first: str, second: str) -> bool:
    return sorted(first) == sorted(second)
```

- Time: `O(n log n)`

- Space: typically `O(n)` in Python

The hashing solution is usually better asymptotically.

Real System Connection

Anagrams themselves are mostly an interview problem, but the underlying pattern appears in:

- Comparing two collections as multisets
- Verifying whether event categories and counts match
- Comparing expected versus actual token distributions
- Detecting equivalent collections regardless of order

Pitfalls

- Checking only whether both strings contain the same unique characters.

For example:

```
"aab" and "abb"
```

Both contain `a` and `b`, but their frequencies differ.

- Forgetting to compare lengths.
- Ignoring case normalization:

```
"Listen" and "Silent"
```

- Ignoring spaces or punctuation when the requirement says to ignore them.
- Assuming only lowercase English letters unless stated.
- Using a fixed array of size 26 when Unicode characters may be present.

---

11. Pattern Comparison

Problem signal	**Pattern**	**Data structure**
Process every element	Traversal	None
Count occurrences	Frequency counting	Dictionary
Check whether seen before	Duplicate detection	Set
Find a previous partner	Complement lookup	Dictionary
Compare unordered counts	Frequency comparison	Dictionary
Answer repeated range sums	Prefix sum	Array
Process continuous portion	Subarray/substring thinking	Depends on problem

---

12. Interview Problem-Solving Framework

For an array, string, or hashing question, use this narration.

Step 1: Clarify

Ask:

- Can the input be empty?
- Can numbers be negative?
- Are duplicates allowed?
- Should I return indexes, values, or a boolean?
- Is the range inclusive?
- Is the input sorted?
- Should comparison be case-sensitive?
- Can I modify the input?

Step 2: Explain Brute Force

Example:

> "I can compare every pair in $O(n^2)$ time and $O(1)$ extra space."

This shows that you understand the basic solution.

Step 3: Identify Repeated Work

Examples:

- Repeatedly searching the earlier array
- Recalculating the same range sum
- Recounting characters
- Comparing every possible pair

Step 4: Select a Pattern

- Repeated membership lookup → set
- Need previous index → dictionary
- Need occurrence counts → frequency map
- Repeated range totals → prefix sum

Step 5: State Complexity Before Coding

Example:

> "I will traverse the array once and store previously seen numbers in a dictionary. The expected time is $O(n)$ and space is $O(n)$."

Step 6: Test Edge Cases

Always test:

```
[]
[5]
duplicate values
negative values
target not found
range beginning at zero
all values identical
```

---

13. Python Templates to Remember

Frequency Template

```
frequencies: dict[str, int] = {}

for item in items:
    frequencies[item] = frequencies.get(item, 0) + 1
```

Seen Set Template

```
seen = set()

for item in items:
    if item in seen:
        return True

    seen.add(item)
```

Complement Template

```
seen = {}

for index, value in enumerate(nums):
    complement = target - value

    if complement in seen:
        return [seen[complement], index]

    seen[value] = index
```

Prefix Sum Template

```
prefix = [0] * (len(nums) + 1)

for index, value in enumerate(nums):
    prefix[index + 1] = prefix[index] + value

range_total = prefix[right + 1] - prefix[left]
```

---

14. Interview Q&A

Question 1: Contains Duplicate

Given an integer array, return `True` if any value appears at least twice.

```
Input: [1, 2, 3, 1]
Output: True
```

Answer

```
def contains_duplicate(nums: list[int]) -> bool:
    seen: set[int] = set()

    for value in nums:
        if value in seen:
            return True

        seen.add(value)

    return False
```

Reasoning

Store previously encountered values in a set. Finding an existing value proves there is a duplicate.

- Time: `O(n)` average
- Space: `O(n)`

Question 2: Two Sum

Return the indexes of two numbers whose sum equals the target.

```
Input: nums = [3, 2, 4], target = 6
Output: [1, 2]
```

Answer

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen: dict[int, int] = {}

    for index, value in enumerate(nums):
        required = target - value

        # The required number must come from an earlier index.
        if required in seen:
            return [seen[required], index]

        seen[value] = index

    return []
```

Reasoning

For every value, search for its required complement among previously seen values.

- Time: `O(n)` average
- Space: `O(n)`

Question 3: Valid Anagram

Determine whether two strings are anagrams.

```
Input: "anagram", "nagaram"
Output: True
```

Answer

```
def is_anagram(first: str, second: str) -> bool:
    if len(first) != len(second):
        return False

    counts: dict[str, int] = {}

    for character in first:
        counts[character] = counts.get(character, 0) + 1

    for character in second:
        if counts.get(character, 0) == 0:
            return False

        counts[character] -= 1

    return True
```

Reasoning

Anagrams must contain identical character frequencies.

- Time: `O(n)`
- Space: `O(k)`

Question 4: Range Sum Query

Given an immutable array, answer multiple queries asking for the sum from `left` to `right`, inclusive.

```
nums = [2, 4, 1, 3]
query(1, 3) = 8
```

Answer

```
class RangeSum:
    def __init__(self, nums: list[int]) -> None:
        self.prefix = [0] * (len(nums) + 1)

        for index, value in enumerate(nums):
            self.prefix[index + 1] = self.prefix[index] + value

    def query(self, left: int, right: int) -> int:
        # right + 1 includes nums[right].
        return self.prefix[right + 1] - self.prefix[left]
```

Reasoning

Precompute cumulative sums once. Every query then becomes one subtraction.

- Construction: `O(n)`
- Each query: `O(1)`
- Space: `O(n)`

Question 5: First Non-Repeating Character

Return the index of the first character that appears exactly once.

```
Input: "leetcode"
Output: 0
```

```
Input: "loveleetcode"
Output: 2
```

Answer

```
def first_unique_character(text: str) -> int:
    counts: dict[str, int] = {}

    # First traversal: count all characters.
    for character in text:
        counts[character] = counts.get(character, 0) + 1

    # Second traversal: preserve original string order.
    for index, character in enumerate(text):
        if counts[character] == 1:
            return index

    return -1
```

Reasoning

The dictionary determines uniqueness. The second traversal finds the earliest unique character.

- Time: `O(n)`
- Space: `O(k)`

Question 6: Most Frequent Event

Return the event that appears most frequently.

```
Input: ["success", "failed", "success", "timeout", "success"]
Output: "success"
```

Answer

```python
def most_frequent_event(events: list[str]) -> str | None:
    if not events:
        return None

    counts: dict[str, int] = {}
    most_frequent = events[0]
    highest_count = 0

    for event in events:
        counts[event] = counts.get(event, 0) + 1

        if counts[event] > highest_count:
            highest_count = counts[event]
            most_frequent = event

    return most_frequent
```

Reasoning

Update the frequency and current maximum during the same traversal.

- Time: $O(n)$
- Space: $O(k)$

An interviewer may ask how ties should be handled. Clarify whether to return:

- First encountered
- Lexicographically smallest
- All tied events

Question 7: Intersection of Two Arrays

Return the unique values present in both arrays.

```
Input: [1, 2, 2, 1], [2, 2]
Output: [2]
```

Answer

```
def intersection(first: list[int], second: list[int]) -> list[int]:
    first_values = set(first)
    result: set[int] = set()

    for value in second:
        if value in first_values:
            result.add(value)

    return list(result)
```

Reasoning

A set provides fast membership checks and automatically removes duplicate results.

- Time: `O(n + m)` average
- Space: `O(n + m)` in the worst case

Question 8: Subarray Sum Equals `k`

Count continuous subarrays whose sum equals k.

```
Input: nums = [1, 1, 1], k = 2
Output: 2
```

The matching subarrays are:

```
indexes 0..1
indexes 1..2
```

Answer

```
def count_subarrays_with_sum(nums: list[int], target: int) -> int:
    """
    Count subarrays whose sum equals target.

    prefix_frequencies[p] tells us how many previous positions
    had prefix sum p.
    """

    prefix_frequencies: dict[int, int] = {
        0: 1  # One empty prefix exists before processing any values.
    }

    running_sum = 0
    matching_subarrays = 0

    for value in nums:
        running_sum += value

        # If an earlier prefix was running_sum - target,
        # removing that earlier part leaves a subarray of sum target.
        required_prefix = running_sum - target
        matching_subarrays += prefix_frequencies.get(required_prefix, 0)

        # Store the current prefix after checking.
        prefix_frequencies[running_sum] = (
            prefix_frequencies.get(running_sum, 0) + 1
        )

    return matching_subarrays


print(count_subarrays_with_sum([1, 1, 1], 2))  # 2
```

Reasoning

Suppose:

```
current prefix sum - earlier prefix sum = target
```

Then:

```
earlier prefix sum = current prefix sum - target
```

Store previous prefix sums in a frequency dictionary.

- Time: `O(n)` average
- Space: `O(n)`

Important Pitfall

Initialize:

```
{0: 1}
```

This represents the empty prefix and allows subarrays beginning at index `0` to be counted.

Also, perform the lookup before inserting the current prefix sum. Otherwise, especially when `target == 0`, you may incorrectly count an empty subarray.

Question 9: Same Event Distribution

Determine whether two lists contain the same events with the same frequencies, regardless of order.

```
first  = ["start", "stop", "start"]
second = ["start", "start", "stop"]

Output: True
```

Answer

```
def same_event_distribution(
    first: list[str],
    second: list[str],
) -> bool:
    if len(first) != len(second):
        return False

    first_counts: dict[str, int] = {}
    second_counts: dict[str, int] = {}

    for event in first:
        first_counts[event] = first_counts.get(event, 0) + 1

    for event in second:
        second_counts[event] = second_counts.get(event, 0) + 1

    return first_counts == second_counts
```

Reasoning

This is the generalized form of anagram checking: compare frequency maps instead of character positions.

- Time: `O(n + m)`
- Space: `O(k)`

---

15. Common Interview Mistakes

Arrays and Strings

- Confusing index and value.
- Using an invalid adjacent index.

- Forgetting empty or one-element inputs.
- Confusing subarray, substring, and subsequence.
- Repeatedly slicing when indexes would be sufficient.
- Forgetting that strings are immutable.

Hashing

- Using a list for repeated membership checks.
- Choosing a set when counts or indexes are required.
- Accessing a missing dictionary key directly.
- Forgetting normalization requirements.
- Assuming output order from a set.
- Not discussing average-case hashing complexity.

Prefix Sums

- Off-by-one errors.
- Forgetting the leading zero.
- Mixing inclusive and exclusive boundaries.
- Using the wrong range formula.
- Forgetting `{0: 1}` in prefix-sum frequency problems.
- Updating the current prefix frequency before performing the lookup.

Two Sum

- Using one element twice.
- Returning values when indexes are requested.
- Mishandling duplicate values.
- Sorting and losing original indexes.
- Not returning immediately after finding a valid pair.

---

16. Day 9 Final Recall Sheet

Pattern 1: Traversal

```
Need to inspect every element?
Perform one left-to-right scan.
```

Typical complexity:

```
Time O(n), space O(1)
```

Pattern 2: Frequency Counting

```
Need “how many times”?
Use dict[value] = count.
```

Typical complexity:

```
Time O(n), space O(k)
```

Pattern 3: Duplicate Detection

```
Need “have I seen this before”?
Use a set.
```

Typical complexity:

```
Time O(n), space O(n)
```

Pattern 4: Complement Lookup

```
Need another value that completes a target?
required = target – current
```

Store previous values in a dictionary.

Typical complexity:

```
Time O(n), space O(n)
```

Pattern 5: Prefix Sum

```
Need repeated continuous range sums?
Store cumulative totals.
```

Formula:

```
sum(left...right) = prefix[right + 1] – prefix[left]
```

Typical complexity:

```
Build O(n)
Query O(1)
Space O(n)
```

Pattern 6: Prefix Sum + Hashing

```
Need to count subarrays with a specific sum?
Store frequencies of previous prefix sums.
```

Formula:

```
required_prefix = running_sum – target
```

Day 9 Interview Takeaway

The most important learning from Day 9 is that dictionaries and sets replace repeated searching.

A brute-force solution often does this:

```
For each element:
    Search all earlier elements
```

That commonly produces $O(n^2)$ time.

A hashing solution does this:

```
For each element:
    Look up earlier information in O(1) average time
```

That commonly reduces the solution to $O(n)$ time.

For interview recall:

> **Need a count? Dictionary. Need existence? Set. Need a partner? Complement lookup. Need repeated range totals? Prefix sum. Need subarray sum counts? Prefix sum plus dictionary.**